

Guia Completo de Estudo: Projeto Físico de Banco de Dados no MySQL

Este guia foi elaborado a partir da análise do material "Aula 5 - Projeto Físico - MySQL" e complementado com pesquisas sobre as melhores práticas oficiais e de mercado para o desenvolvimento e manutenção de bancos de dados relacionais. O objetivo é fornecer um material rico em detalhes, focado não apenas nos comandos, mas também no "porquê" das escolhas arquiteturais.

1. O Que é o Projeto Físico?

O projeto físico representa a etapa final e mais concreta na modelagem de um banco de dados. Após a definição do modelo conceitual (entidades e relacionamentos) e do modelo lógico (tabelas e chaves), o projeto físico traduz essas estruturas para a linguagem específica de um Sistema Gerenciador de Banco de Dados (SGBD), neste caso, o **MySQL**.

Nesta fase, o foco principal é a utilização da **Data Definition Language (DDL)**, um subconjunto da linguagem SQL responsável por definir, alterar e remover as estruturas que armazenarão os dados. Diferente da DML (Data Manipulation Language), que lida com os dados em si (INSERT, UPDATE, DELETE), a DDL lida com o "esqueleto" do banco.

2. A Importância da Escolha dos Tipos de Dados

Uma das decisões mais críticas no projeto físico é a escolha correta dos tipos de dados para cada coluna. Uma escolha inadequada pode resultar em desperdício de espaço em disco, lentidão nas consultas e até mesmo perda de precisão dos dados ¹. A regra de ouro, segundo a documentação oficial do MySQL, é: **use sempre o tipo mais preciso e menor possível que atenda aos requisitos do negócio** ².

Tipos Numéricos

Os tipos numéricos são divididos em inteiros, decimais de precisão fixa e decimais de ponto flutuante.

Tipo de Dado	Descrição e Uso Recomendado
INT ou INTEGER	Utilizado para números inteiros padrão. Ocupa 4 bytes.
MEDIUMINT	Uma alternativa mais leve ao INT quando o intervalo de valores não é tão grande (ex: até 8 milhões sem sinal). Ocupa 3 bytes,

	economizando 25% de espaço em relação ao <code>INT</code> 2 .
<code>DECIMAL(m, d)</code>	Essencial para valores monetários ou dados científicos onde a precisão exata é obrigatória. O parâmetro <code>m</code> define o total de dígitos e <code>d</code> as casas decimais. Evita os erros de arredondamento comuns em tipos de ponto flutuante.
<code>FLOAT</code> e <code>DOUBLE</code>	Tipos de ponto flutuante. São mais rápidos para cálculos matemáticos complexos, mas não garantem precisão absoluta. Devem ser evitados para dinheiro.

Tipos de Texto (Strings)

A escolha entre `CHAR` e `VARCHAR` é um ponto de discussão frequente entre desenvolvedores e DBAs.

Tipo de Dado	Descrição e Uso Recomendado
<code>CHAR(n)</code>	String de tamanho fixo. Se você definir <code>CHAR(10)</code> e inserir "A", o MySQL preencherá os 9 espaços restantes. Melhor prática: Usar para dados que sempre têm o mesmo tamanho, como siglas de estados (UF), CEPs, CPFs ou hashes de senhas. É mais rápido para o motor do banco de dados processar do que o <code>VARCHAR</code> 3 .
<code>VARCHAR(n)</code>	String de tamanho variável. Ocupa apenas o espaço do texto inserido mais 1 ou 2 bytes para registrar o comprimento. Ideal para nomes, endereços e e-mails.
<code>TEXT</code>	Utilizado para grandes volumes de texto, como artigos de blog ou descrições longas.

Tipos de Data e Hora

O MySQL oferece tipos específicos para lidar com o tempo, o que facilita cálculos (como "adicionar 30 dias") e formatações.

Tipo de Dado	Descrição e Uso Recomendado
DATE	Armazena apenas a data no formato YYYY-MM-DD .
DATETIME	Armazena data e hora combinadas.
TIMESTAMP	Semelhante ao DATETIME , mas é frequentemente usado para registrar automaticamente o momento exato em que uma linha foi criada ou atualizada.

Tipos Especiais

O tipo `ENUM` é particularmente interessante. Ele permite definir uma lista estática de valores permitidos para uma coluna (ex: `ENUM('M', 'F')` para sexo). Historicamente, o MySQL não suportava a restrição `CHECK` (que valida dados antes da inserção), então o `ENUM` era a principal forma de garantir que apenas valores específicos fossem aceitos no nível do banco de dados. Vale notar que a partir da versão 8.0.16, o MySQL passou a suportar a restrição `CHECK` nativamente ² .

3. Comandos DDL Essenciais

A construção do banco de dados é feita através de comandos DDL. Abaixo estão os principais comandos discutidos no material, com suas aplicações práticas.

Criação e Seleção do Banco

Antes de criar tabelas, é necessário criar o contêiner principal:

```
CREATE DATABASE nome_do_banco;
```

```
USE nome_do_banco;
```

Criação de Tabelas (`CREATE TABLE`)

O comando `CREATE TABLE` é o coração do projeto físico. Ele define o nome da tabela, suas colunas, os tipos de dados e as restrições (constraints).

Exemplo prático de criação de uma tabela de clientes:

```
SQL
```

```
CREATE TABLE cliente (  
    codigoCliente INT AUTO_INCREMENT,  
    Nome VARCHAR(50) NOT NULL,  
    CEP CHAR(8) NOT NULL,  
    CONSTRAINT pk_cliente PRIMARY KEY (codigoCliente)  
) ENGINE=InnoDB;
```

A cláusula `ENGINE=InnoDB` é crucial. O InnoDB é o motor de armazenamento padrão do MySQL e suporta transações (ACID) e chaves estrangeiras, garantindo a integridade dos dados ².

Alteração de Tabelas (`ALTER TABLE`)

Os requisitos de negócio mudam, e o banco de dados precisa se adaptar. O comando `ALTER TABLE` permite modificar a estrutura sem perder os dados existentes.

- **Adicionar coluna:** `ALTER TABLE pessoa ADD idade INTEGER;`
- **Modificar tipo de dado:** `ALTER TABLE pessoa MODIFY nome VARCHAR(40);`
- **Renomear coluna:** `ALTER TABLE pessoa CHANGE nome nome_completo VARCHAR(50);`
- **Remover coluna:** `ALTER TABLE pessoa DROP COLUMN idade;`
- **Adicionar Chave Estrangeira:** `ALTER TABLE cliente ADD CONSTRAINT fk_loja FOREIGN KEY (id_loja) REFERENCES loja (id_loja);`

Remoção de Tabelas (`DROP` e `TRUNCATE`)

Existem duas formas principais de remover dados em massa, mas elas têm propósitos diferentes:

- `DROP TABLE nome_tabela;` : Apaga a tabela inteira, incluindo sua estrutura e todos os dados. **Atenção:** Se a tabela fornecer uma chave primária para outra tabela (relacionamento), o MySQL bloqueará o `DROP` para evitar dados órfãos. É necessário remover a *constraint* de chave estrangeira primeiro.
- `TRUNCATE TABLE nome_tabela;` : Remove todos os registros da tabela de forma extremamente rápida, mas **mantém a estrutura** (colunas e restrições) intacta. É ideal para limpar tabelas temporárias ou de log.

4. Garantindo a Qualidade dos Dados: Constraints

As *Constraints* (Restrições) são regras aplicadas às colunas para garantir a integridade e a confiabilidade dos dados. Elas impedem a inserção de dados inválidos.

Constraint	Função e Importância
PRIMARY KEY	Define o identificador único de cada registro na tabela. Nenhuma linha pode ter a mesma chave primária, e ela não pode ser nula.
FOREIGN KEY	Cria um vínculo entre duas tabelas, garantindo a Integridade Referencial . Impede, por exemplo, que um pedido seja criado para um cliente que não existe.
NOT NULL	Obriga que a coluna seja preenchida. Essencial para campos obrigatórios como CPF ou Nome.
UNIQUE	Garante que todos os valores na coluna sejam diferentes (ex: e-mails de usuários), mas, diferente da Primary Key, permite valores nulos (dependendo da configuração).
DEFAULT	Insere um valor padrão automaticamente se o usuário não fornecer um. Exemplo: <code>DEFAULT 'Ativo'</code> para um status de cadastro.

5. Funções de Apoio e Metadados

O MySQL fornece comandos úteis para inspecionar a estrutura do banco de dados, o que é vital durante o desenvolvimento e a manutenção.

- `SHOW TABLES;` : Lista todas as tabelas do banco de dados selecionado.
- `DESCRIBE nome_tabela;` : Exibe a estrutura detalhada da tabela (colunas, tipos, chaves).
- `SHOW CREATE TABLE nome_tabela;` : Uma ferramenta poderosa que retorna o comando SQL exato que foi utilizado para criar a tabela, incluindo todas as configurações do motor de armazenamento.

6. Manutenção e Performance (Tópico Avançado)

Além da criação, o projeto físico deve prever a manutenção a longo prazo. Conforme os dados são inseridos, atualizados e deletados, o banco de dados pode sofrer com **fragmentação**, o que degrada a performance ⁴.

- **Desfragmentação:** O comando `OPTIMIZE TABLE` reorganiza o armazenamento físico dos dados e dos índices, recuperando espaço não utilizado e melhorando a velocidade de

leitura 4 . Recomenda-se o uso quando a fragmentação ultrapassa 35%.

- **Estatísticas:** O MySQL coleta estatísticas sobre a distribuição dos dados para decidir a melhor forma de executar uma consulta (Plano de Execução). Manter essas estatísticas atualizadas é fundamental para a performance, embora a atualização constante possa causar recompilações excessivas de consultas 4 .

Referências

[1] Malavasi, A. "25 dicas e boas práticas de banco de dados para desenvolvedores". Medium.

[2] MySQL Documentation. "Chapter 13 Data Types". Oracle. Disponível em:

[3] Rivercrane. "Top 10 Best Practices in MySQL".

[4] Bomfim, J. (2022). "Melhores práticas para os bancos de dados MySQL – Manutenção Automática". DBA. Disponível em: